Course Discussions

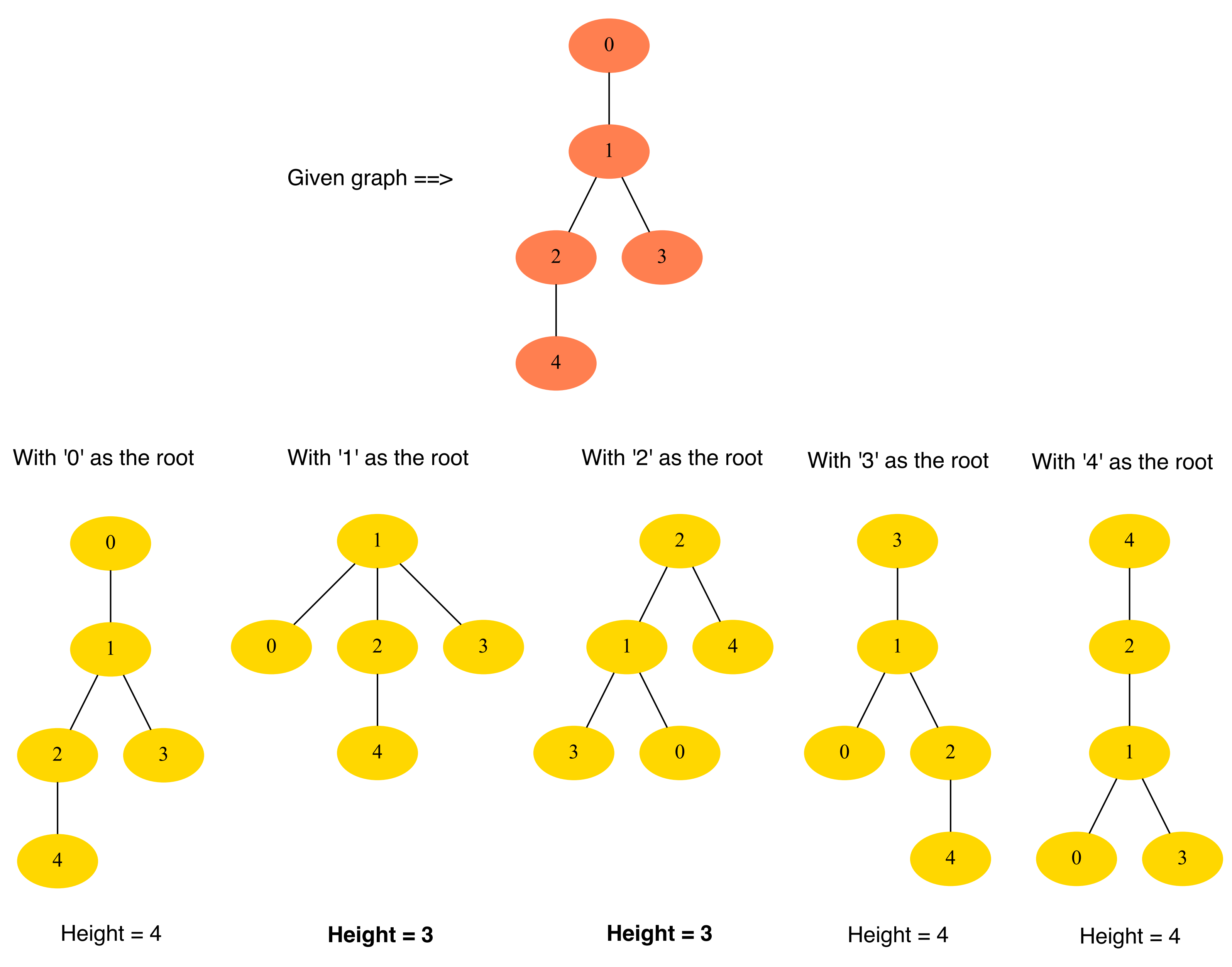
Solution: Problem Challenge 2: Minimum Height Trees

Problem Statement

We are given an undirected graph that has the characteristics of a k-ary tree. In such a graph, we can choose any node as the root to make a k-ary tree. The root (or the tree) with the minimum height will be called Minimum Height Tree (MHT). There can be multiple MHTs for a graph. In this problem, we need to find all those roots which give us MHTs. Write a method to find all MHTs of the given graph and return a list of their roots.

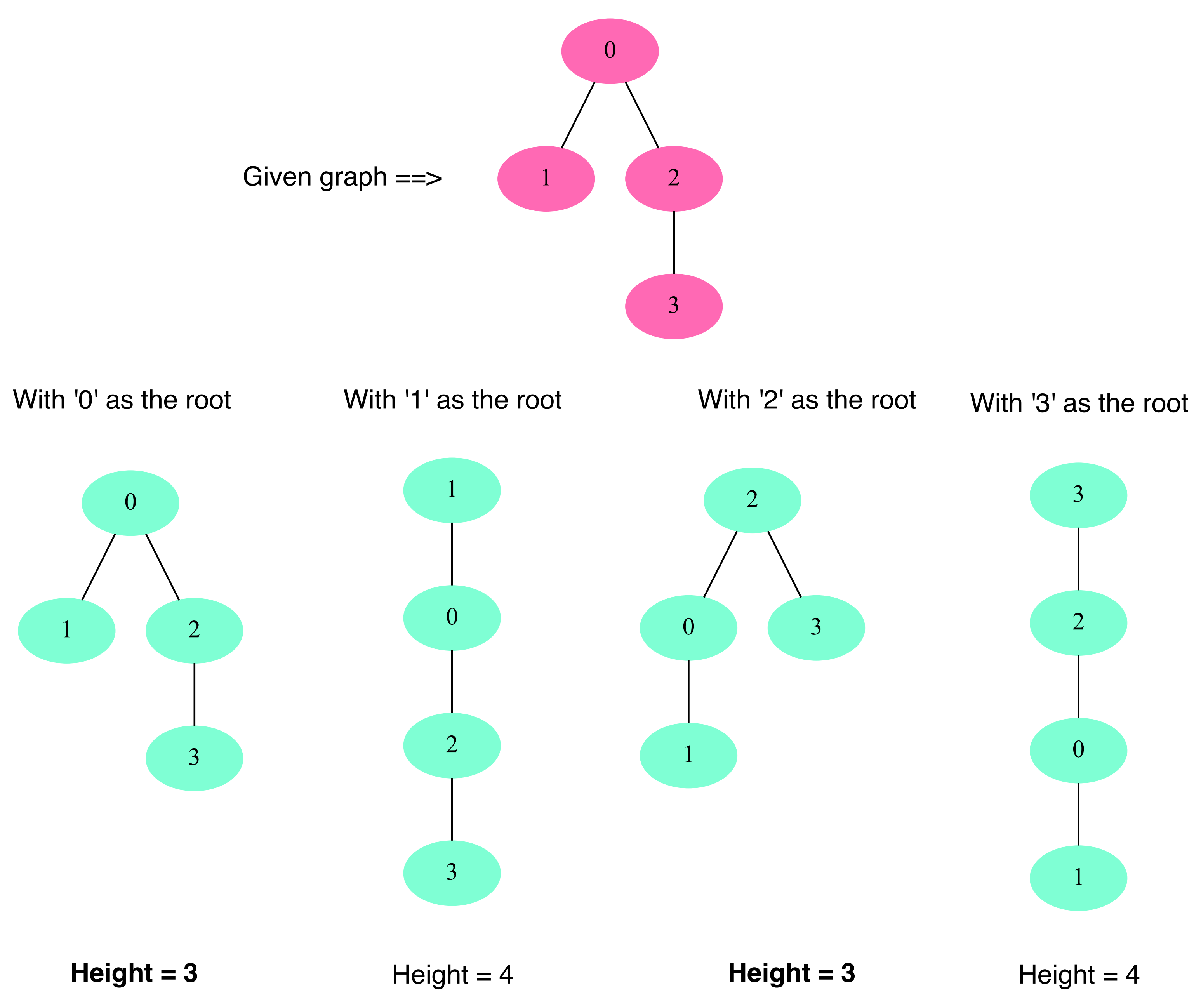
Example 1:

```
Input: vertices: 5, Edges: [[0, 1], [1, 2], [1, 3], [2, 4]]
Output:[1, 2]
Explanation: Choosing '1' or '2' as roots give us MHTs. In the below
diagram, we can see that the height of the trees with roots '1' or '2'
is three which is the minimum.
```



Example 2:

```
Input: vertices: 4, Edges: [[0, 1], [0, 2], [2, 3]]
Output:[0, 2]
Explanation: Choosing '0' or '2' as roots give us MHTs. In the below
diagram, we can see that the height of the trees with roots '0' or '2'
is three which is minimum.
```



Example 3:

```
Input: vertices: 4, Edges: [[0, 1], [1, 2], [1, 3]]
Output:[1]
```

Solution

The key intuition behind solving this problem is based on the definition of a tree's height: the height of a tree is the number of edges on the longest path between the root and any leaf. So, an MHT is a tree that minimizes this longest path.

Imagine we have a longest path P in the tree. The path P has two ends; let's call them end A and end B. Now, let's consider what the root of an MHT can be:

- If we select a root that is not on the path P, the height of the tree would at least be the length of P, because there would be a path from the root to either A or B that is longer than P (as it includes P plus some additional edges). Therefore, the root of the MHT must be on P.
- If the root is on P, but not in the middle of P, then the height of the tree will be larger than if we selected the root in the middle of P, because the longest path will be from the root to either end of P. Therefore, the root of the MHT must be in the middle of P.

So, the problem of finding the MHT root(s) reduces to finding the middle node(s) of the longest path in the tree.

We can find the middle node(s) of the longest path by using an algorithm called 'leaf pruining'. Let's look into this.

From the above discussion, we can deduce that the leaves can’t give us MHT, hence, we can remove them from the graph and remove their edges too. Once we remove the leaves, we will have new leaves. Since these new leaves can’t give us MHT, we will repeat the process and remove them from the graph too. We will prune the leaves until we are left with one or two nodes which will be our answer and the roots for MHTs.

The algorithm works because when you trim leaves, you're essentially trimming the ends of all the longest paths in the tree. If there's one longest path, you're trimming it from both ends, and if there are multiple longest paths, you're trimming them all. Eventually, you're left with one or two nodes, which must be the middle of the longest path(s), and those are the roots of the MHTs.

We can implement the above process using the topological sort. Any node with only one edge (i.e., a leaf) can be our source and, in a stepwise fashion, we can remove all sources from the graph to find new sources. We will repeat this process until we are left with one or two nodes in the graph, which will be our answer.

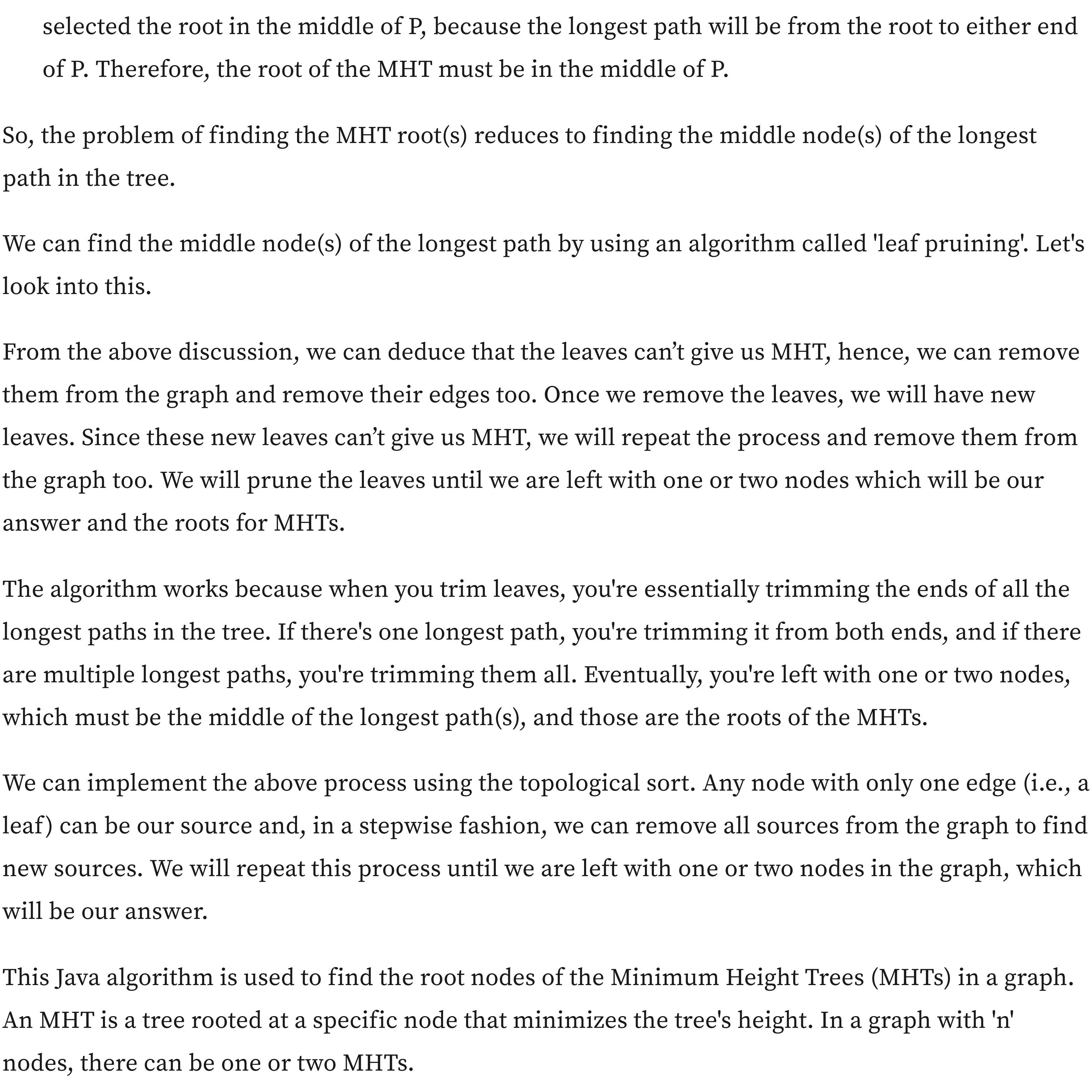
This Java algorithm is used to find the root nodes of the Minimum Height Trees (MHTs) in a graph. An MHT is a tree rooted at a specific node that minimizes the tree's height. In a graph with 'n' nodes, there can be one or two MHTs.

Here's a breakdown of the algorithm:

1. It starts by checking if the number of nodes is less than or equal to 0, returning an empty list if true, as there would be no trees in the graph. If the graph contains only one node, it returns that single node as an MHT.
2. Next, it initializes two HashMaps, `inDegree` to store the count of incoming edges for every vertex and `graph` as an adjacency list representation of the graph. It populates these HashMaps with initial values.
3. The algorithm then constructs the graph. As it's an undirected graph, each edge connects two nodes bi-directionally, meaning it adds a link for both nodes and increments the in-degrees of the two nodes.
4. The algorithm finds all leaf nodes (nodes with only one in-degree) and adds them to a queue.
5. Next, it iteratively removes the leaf nodes level by level, subtracting one from the in-degree of each leaf node's children. If a child node becomes a leaf node as a result, it is added to the queue of leaf nodes. This process repeats until the graph has been reduced to one or two nodes, which represent the roots of the MHTs.
6. Finally, the algorithm adds the remaining nodes in the leaves queue to `minHeightTrees` and returns this list. These nodes are the roots of the MHTs in the graph.

Code

Here is what our algorithm will look like:



```
import java.util.*;

class Solution {
  public List<Integer> findTrees(int nodes, int[][] edges) {
    List<Integer> minHeightTrees = new ArrayList<>();
    if (nodes <= 0)
      return minHeightTrees;

    // with only one node, since its in-degree will be 0, therefore, we need to handle
    // it separately
    if (nodes == 1) {
      minHeightTrees.add(0);
      return minHeightTrees;
    }

    // a. Initialize the graph
    HashMap<Integer, Integer> inDegree =
        new HashMap<>(); // count of incoming edges for every vertex
    HashMap<Integer, List<Integer>> graph = new HashMap<>(); // adjacency list graph
    for (int i = 0; i < nodes; i++) {
      inDegree.put(i, 0);
```

You must run your code first

Time Complexity

In step ‘d’, each node can become a source only once and each edge will be accessed and removed once. Therefore, the time complexity of the above algorithm will be O(V+E), where ‘V’ is the total nodes and ‘E’ is the total number of the edges.

Space Complexity

The space complexity will be O(V+E), since we are storing all of the edges for each node in an adjacency list.

← Previous
Problem Challenge 2: Minimum Height Trees (Hard)

Next →
Introduction To Union Find Pattern

☐ Mark as Completed